

MODELS | AGENTS | ORGANIZATION | GOVERNANCE

RECURSIVE ORGANIZATIONAL ADAPTATION

From fixed trading models to self-reconfiguring AI systems

A didactic guide to building trading systems that can diagnose their own organizational limitations, propose bounded specialist capabilities, validate those changes, and adapt under an immutable governance layer.

THE QUESTION THAT ORGANIZES THIS PAPER

Traditional algorithmic trading asks how to improve a signal, a model, or a set of parameters. This paper asks a deeper question: *what if the organization that produces the trading decision is itself allowed to adapt?* The companion notebook constructs a controlled laboratory in which market regimes change, a fixed agent organization encounters different failure modes, a bounded language-model supervisor proposes specialist capabilities, a deterministic sandbox tests those proposals, and a constitutional layer determines what may actually be deployed. The central idea is not uncontrolled self-replication. It is governed, evidence-based organizational adaptation.

Alejandro Reynoso

September 2026

Pedagogical Companion Paper

Algorithmic Trading / Artificial Intelligence / Governance

Abstract

Algorithmic trading education traditionally moves from market data to features, from features to forecasts, and from forecasts to positions. That sequence is still essential, but artificial intelligence introduces a higher-order possibility: the system that organizes forecasting, risk control, execution, and monitoring can itself become adaptive. This paper develops that idea through the companion Colab notebook on *Recursive Organizational Adaptation*. The notebook does not ask a language model to predict prices directly. Instead, it creates a synthetic market with changing regimes, constructs a small trading organization of specialized deterministic agents, measures where that fixed organization fails, and then introduces a bounded language-model supervisor whose task is to reason about organizational adequacy rather than to trade.

The resulting architecture separates several responsibilities that are often blurred together in discussions of “AI trading.” The Signal Agent produces a directional view, the Risk Agent scales exposure, the Execution Agent accounts for implementation costs, and the Supervisor Agent studies whether the organization itself is missing a capability. Three pre-approved specialists – volatility, liquidity, and mean reversion – can be activated when the evidence supports them. The language model may propose a change, but it cannot execute arbitrary code, alter leverage limits, rewrite the constitution, or deploy a specialist directly. Every proposed reconfiguration must first compete against alternatives in a deterministic sandbox. A governance gate authorizes deployment only when the measured improvement exceeds a minimum threshold and the proposal remains within institutional constraints.

The notebook therefore offers a concrete bridge from traditional quantitative models to self-improving AI systems. Its form of self-improvement is deliberately bounded: the underlying language model does not retrain itself, the quantitative agents do not rewrite their mathematics, and the constitution remains immutable. What changes is the *organization of capabilities*. A volatility specialist may be added when fast-moving risk invalidates a slower architecture; a liquidity specialist may slow rebalancing when spreads widen; a mean-reversion specialist may alter signal composition when return dependence reverses. Equally important, specialists can be retired when they are no longer justified. Improvement is thus understood as evidence-based organizational learning rather than permanent accumulation of complexity.

This paper is written for students of Algorithmic Trading who are taking the next step from individual models toward systems that use artificial intelligence. It explains every conceptual stage of the notebook in fluid, non-code prose, interprets the logic of the synthetic environment, motivates the organizational utility function, examines the role of Claude Sonnet 5 as a bounded supervisor, and develops the governance principles required to keep recursion observable, reversible, and auditable. The broader claim is that the future unit of analysis in quantitative finance may no longer be the model alone. It may be the governed organization of models and agents that decides which capabilities should exist, when they should be active, and when the institution should return to a simpler form.

How to read this paper

The intended reader already understands basic ideas such as returns, volatility, momentum, transaction costs, drawdown, and backtesting, but need not have studied multi-agent systems.

The paper begins by explaining why ordinary model adaptation is not the same thing as organizational adaptation. It then follows the notebook in sequence, from the synthetic market through the final comparison between a fixed and a recursively adaptive organization. Each code-cell chapter explains not only *what* the cell computes but *why the computation exists in the architecture*. Nine figure locations correspond to the infographics already designed for Cells 2 through 10. If the image files are absent, the document compiles with informative placeholders so that it can be uploaded to Prism or another LaTeX environment before the figures are added.

A small number of equations are included because the notebook contains quantitative rules that students should be able to recognize. They are always interpreted in words. The paper deliberately avoids turning the discussion into a sequence of bullets. The objective is to make the full architecture intelligible as a coherent system: environment, organization, diagnosis, proposal, experiment, governance, deployment, monitoring, and recursive adaptation.

Contents

Abstract	2
1 The intellectual step beyond traditional algorithmic trading	6
2 From self-improvement to organizational adaptation	7
3 The complete architecture before we enter the notebook	8
4 Cell 1: Establishing the laboratory and the boundary between AI and code	9
5 Cell 2: Building a market in which organizational adaptation has a reason to exist	9
6 Cell 3: Inspecting the environment before changing the organization	12
7 Cell 4: Defining the trading organization, its specialist registry, and its constitution	14
8 Cell 5: Why a fixed architecture must be allowed to fail	16
9 Cell 6: Turning market evidence into an organizational diagnostic state	18
10 Cell 7: Claude Sonnet 5 as a bounded organizational supervisor	20
11 Cell 8: The sandbox as the experimental court of appeal	22
12 Cell 9: The full recursive organizational adaptation loop	24
13 Cell 10: Comparing the fixed institution with the adaptive institution	26
14 What, exactly, is improving?	28
15 The two-layer institution: adaptive layer and constitutional layer	28
16 Organizational utility: why the objective must include complexity	29
17 The Supervisor as a meta-agent	30
18 Why the sandbox is more important than the language model	30
19 Failure modes: how a self-improving organization can go wrong	31
20 From the classroom market to real algorithmic trading	32
21 A deeper interpretation: the organization becomes a state variable	33
22 The audit ledger as institutional memory	34
23 Why this is not simply regime switching	34
24 Connecting Recursive Organizational Adaptation to STOP	35
25 The classroom journey: from quantitative model to governed AI institution	35
26 Research extensions for a second-generation notebook	36
27 Limits of the current notebook	37

28 Conclusion: from trading algorithms to adaptive financial organizations	38
A Appendix A: A compact glossary for students	40
B Appendix B: Notebook-to-paper map	41
C Appendix C: Figure file map for Prism/OpenAI LaTeX editor	41
References	43

1 The intellectual step beyond traditional algorithmic trading

A student usually encounters algorithmic trading as a progression of increasingly sophisticated models. One begins with a rule such as moving-average crossover, then learns regression, factor models, volatility estimation, machine learning, neural networks, portfolio construction, and execution. Every new technique appears to expand the menu of available predictors or decision rules. This model-centric progression is useful, but it leaves one element relatively fixed: the architecture that decides which capability should be used, how the capabilities interact, and what should happen when the market changes in a way the original architecture did not anticipate.

This distinction becomes more important as soon as we accept that markets are non-stationary. A momentum signal can be useful during persistent trends and damaging during sharp reversal. A risk scaler based on a slow volatility window can be adequate in ordinary markets and too sluggish during a sudden volatility shock. An execution policy calibrated to normal spreads can generate unacceptable costs when liquidity deteriorates. These are not all versions of the same forecasting error. They are different kinds of organizational mismatch. The correct response to a reversal regime may be to alter signal interpretation; the correct response to a volatility shock may be to alter risk scaling; the correct response to liquidity stress may be to alter execution pacing. A single fixed procedure may therefore contain the right components for one environment and the wrong division of labor for another.

The conventional response is to make parameters adaptive. We change a lookback length, retune a threshold, refit a model, or rebalance weights. Recursive Organizational Adaptation moves the object of adaptation one level upward. It asks whether the institution can recognize that its current division of cognitive labor is insufficient and then create, remove, or reroute specialist capabilities. This is not the same as selecting a hyperparameter. It is a change in the structure that produces the decision.

The idea is easiest to understand by analogy with a human trading organization. A desk that encounters a new regulatory requirement may create a compliance function. A portfolio team operating in a new market may add a specialist in local microstructure. A firm that expands into options may need a new risk capability. Human institutions do not respond to every new problem by asking existing employees to adjust one numerical parameter. They reorganize expertise. The notebook asks whether a bounded AI architecture can perform a simplified version of that process.

This leads to a new educational ladder. At the first level, students learn a model. At the second, they learn how several models combine into a strategy. At the third, they learn how specialized agents can divide responsibilities. At the fourth, they learn how an organization of agents can diagnose its own limitations. At the fifth, they learn how proposed reorganization can be tested and governed. Recursive Organizational Adaptation is therefore not a replacement for quantitative finance. It is a systems layer built on top of it.

The central shift is from adapting the model to adapting the organization that decides which model, risk process, and execution capability should be active.

This framing is especially important for students because it prevents an increasingly common misunderstanding: that adding a language model to a trading strategy automatically makes it “AI-native.” The notebook takes the opposite position. The quantitative calculations remain deterministic and inspectable. Artificial intelligence is used where flexible reasoning is valuable: interpreting a compact diagnostic state and proposing a bounded organizational response. The final authority remains outside the language model in numerical validation and governance.

2 From self-improvement to organizational adaptation

The general framework behind this notebook draws on two related streams of ideas. The first is the Self-Taught Optimizer tradition, which emphasizes that capability can improve when the *scaffold* around a fixed model is improved. In STOP, the object of optimization can move from a candidate answer to the procedure that generates and evaluates candidates, and eventually to the improver itself [7]. The important lesson for finance is not that a language model must rewrite its own weights. It is that the system surrounding a model – its search, evaluation, memory, routing, and selection rules – can itself become an object of improvement.

The second inspiration comes from work on autonomous replication, where a functioning agentic system can produce another operational instance and, in the stronger case, allow the new instance to continue the process [1]. The financial application deliberately changes the meaning of replication. In an uncontrolled cybersecurity context, autonomous propagation is a risk. In finance, the useful architectural insight is that a system can instantiate a new capability and transfer a bounded role to it. We therefore replace uncontrolled replication with *permissioned organizational reproduction*. A new specialist is not a free-standing clone with arbitrary powers. It is a role defined in advance, granted a limited interface, evaluated before deployment, and subject to retirement.

This translation clarifies what “recursive” means in the notebook. The organization used in one market regime becomes the starting organization for the next regime. If a volatility specialist is admitted, the next diagnostic cycle begins with that specialist active. If the market later changes and the specialist no longer contributes enough value, the organization can contract. The result of one adaptation is therefore the input state of the next adaptation. That is the recursion.

It is equally important to clarify what is *not* recursive. The language model does not recursively fine-tune itself. The Python source code does not rewrite itself. The constitution does not evolve. The notebook implements a bounded form of self-improvement in which the organizational configuration changes within a fixed action space. This choice is pedagogically powerful because students can see every permitted move. The system is adaptive enough to demonstrate the concept while remaining simple enough to audit.

A useful way to distinguish the layers is to think of three kinds of change. Parameter adaptation changes a number inside a given model. Model adaptation changes which model or model family is used. Organizational adaptation changes which capabilities exist, how responsibilities are distributed, and which components can influence the final decision. The notebook focuses on the third layer while keeping the first two deliberately simple.

A BOUNDED DEFINITION OF SELF-IMPROVEMENT

In this paper, a system is called self-improving when evidence generated by its own operation can cause the system to change the organization of approved capabilities used in subsequent operation. The improvement is bounded because the system may only choose from pre-audited roles, every change is validated externally, and constitutional constraints are not modifiable by the supervisor.

3 The complete architecture before we enter the notebook

The notebook contains a hierarchy of responsibilities. At the bottom is the synthetic market, which generates prices, returns, volatility, persistence, and trading costs. Above the market sit deterministic trading components. The Signal Agent turns recent price behavior into a directional target. The Risk Agent scales that target according to estimated volatility. The Execution Agent determines how quickly the system moves toward the desired position and explicitly accounts for spread costs. These components create a complete, though intentionally simple, trading system.

Above them sits the Supervisor. Its responsibility is qualitatively different. The Supervisor does not forecast the next return and does not submit a position. It evaluates whether the organization is fit for the current environment. To do that, it receives a compact diagnostic state containing variables such as realized volatility, average spread, autocorrelation, momentum, and current performance. It then considers whether the observed problem suggests a missing capability.

The Supervisor cannot invent unlimited capabilities. The notebook defines a specialist registry containing three options. The volatility specialist makes risk scaling more responsive to fast changes in volatility. The liquidity specialist slows rebalancing when spreads are expensive. The mean-reversion specialist changes signal composition when serial dependence becomes sufficiently negative. Each specialist has a narrow domain. The registry therefore acts as a library of pre-audited organizational modules.

A proposal is not a deployment. This separation is one of the central design principles of the notebook. The system first constructs a set of candidate configurations that includes the current organization, potential additions, potential retirements, and selected combinations. Each candidate is then backtested over the observation window. The deterministic evaluator calculates return, volatility, a Sharpe-like risk-adjusted measure, drawdown, turnover, transaction cost, and organizational complexity. These quantities are combined in an organizational utility function.

The configuration with the highest measured utility is the sandbox winner, but even that is not automatically deployed. The governance gate compares the winner with the current organization and requires a minimum improvement. It also applies hard constitutional filters. Only if the candidate improves enough and remains within the allowed organizational space does the system change. Otherwise the existing organization survives.

This produces a disciplined loop:

Observe → Diagnose → Propose → Validate → Govern → Deploy → Observe again

The apparent simplicity of this sequence hides an important philosophical change. A traditional backtest evaluates a strategy. This notebook evaluates both a strategy and the process by which the organization responsible for that strategy can change. The architecture therefore introduces a meta-level without abandoning quantitative discipline.

4 Cell 1: Establishing the laboratory and the boundary between AI and code

The first code cell performs setup, but its role is more significant than installing a software package. It establishes where language-model reasoning enters the experiment. The random seed is fixed so that the synthetic market and deterministic parts of the notebook can be reproduced. The Anthropic client is created only when the Colab secret `ANTHROPIC_API_KEY` is available, and the model identifier is set to `claude-sonnet-5`. If the key is absent, the rest of the numerical architecture can still run through the transparent heuristic supervisor defined later.

This is the first governance lesson. We should be able to identify exactly which outputs are stochastic or model-dependent and which are deterministic. The market simulation is stochastic but reproducible under the seed. Trading calculations are deterministic given the data and configuration. The language model is used for a bounded reasoning task. The governance gate is deterministic. By drawing those boundaries explicitly, the notebook makes it possible to audit the contribution of the LLM rather than treating the entire system as one opaque “AI model.”

The cell also introduces graceful degradation. A production financial system should not become unusable merely because an external model service is unavailable. The fallback path is intentionally simple, but it demonstrates a broader principle: agentic systems should have defined behavior when probabilistic or remote components fail. This is especially relevant in markets, where availability and deterministic contingency behavior can be as important as predictive accuracy.

Finally, the setup cell creates a methodological separation between recommendation and execution. Claude is not given a general-purpose Python execution channel. It will return structured recommendations that are parsed and sanitized. That design anticipates the most important control introduced later: language-model reasoning is advisory; the institution remains the authority.

5 Cell 2: Building a market in which organizational adaptation has a reason to exist

A self-reconfiguring organization cannot teach us much if the environment is stationary. The second code cell therefore creates the reason for adaptation. Instead of simulating one homogeneous return process, it joins six sequential market regimes: Calm, Momentum, HighVol,

LiquidityStress, Reversal, and Recovery. Each lasts 180 business days, producing 1,080 observations in total. The regimes differ in drift, volatility, autoregressive persistence, and typical spread.

The return process is deliberately modest. Within each regime, the daily return is generated from a regime-specific drift, a persistence term linked to the previous return, and a random shock. In compact form,

$$r_t = \mu + \phi r_{t-1} + \varepsilon_t.$$

The notation matters less than the intuition. The parameter μ changes the directional bias, the volatility of ε_t changes the typical size of shocks, and ϕ changes the relationship between consecutive returns. A positive value of ϕ produces persistence that can help a momentum-oriented architecture. A strongly negative value creates reversal behavior that can punish it. The spread is simulated separately with a positive distribution whose typical level changes by regime, allowing execution costs to become a first-class feature of the environment rather than an afterthought.

This design turns a single synthetic price series into six distinct organizational problems. Calm conditions are relatively forgiving. Momentum rewards persistence. HighVol attacks slow risk control. LiquidityStress attacks execution assumptions by raising spreads sharply. Reversal attacks the signal architecture by making recent continuation unreliable. Recovery creates an environment in which the need for some previously useful specialists may disappear. The system therefore encounters not merely six levels of difficulty but six different reasons why a fixed organization might be inadequate.

The code then engineers five observable features. Twenty-day momentum captures intermediate trend. A five-day reversed price change provides a simple mean-reversion feature. Twenty-day and five-day volatility estimates create slow and fast measures of risk. A forty-day lag-one autocorrelation estimates whether return dynamics look persistent or reversing. These features are intentionally interpretable because the notebook is designed to teach organizational reasoning. A student should be able to explain why negative autocorrelation might justify a mean-reversion specialist and why a widening spread might justify an execution specialist.

The true regime label is retained in the data only for classroom evaluation. The trading organization does not need the label to calculate its actions. This matters because real markets do not announce “you are now in Liquidity Stress.” A live system would have to infer changing conditions from observable evidence. The notebook keeps the hidden label so the instructor can later assess whether adaptation happened for the expected reason.

The synthetic environment is therefore not an attempt to reproduce the entire complexity of financial markets. It is a controlled experiment that makes causal interpretation possible. When a specialist improves the organization in one regime, students can connect that improvement to a known change in the data-generating process. This controlled structure is exactly what makes the notebook useful as a first encounter with adaptive organizations.

CELL
2

Creating a Synthetic Market with Six Structural Regimes

We generate a realistic, multi-regime price series with changing dynamics, volatility and transaction costs to test how an adaptive trading organization responds to different market conditions.

A controlled, synthetic environment lets us study organizational adaptation with known ground truth.

DIFFERENT ENVIRONMENTS.
DIFFERENT CHALLENGES. SAME MARKET.

1 Define Market Regimes

We specify six sequential regimes, each with different statistical properties.

Regime	Length (days)	Drift (μ)	Volatility (σ)	AR(1) (ϕ)	Typical Spread (bps)
Calm	180	0.0002	0.006	0.05	2.0
Momentum	180	0.0004	0.009	0.35	3.0
HighVol	180	-0.0001	0.022	0.10	5.0
Liquidity Stress	180	0.0000	0.014	0.02	25.0
Reversal	180	0.0001	0.012	-0.45	4.0
Recovery	180	0.0005	0.009	0.20	3.0

2 Simulate Returns Sequentially

For each regime, we generate returns using an autoregressive process with drift and noise.

$$r_t = \mu + \phi r_{t-1} + \varepsilon_t \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2)$$

- μ Daily drift (average return)
- σ Daily volatility (random shocks)
- ϕ AR(1) persistence (momentum if $\phi > 0$, mean reversion if $\phi < 0$)
- ε_t Random shock from a normal distribution

We also simulate a realistic, positive bid-ask spread for each day using a lognormal distribution:

$$\text{spread}_t \sim \text{LogNormal}(\ln(s), 0.15)$$

where s is the typical spread for the regime (in bps).

3 Result: A Multi-Regime Market

The synthetic price series exhibits distinct behaviors across the six regimes.



4 Regime Characteristics

Each regime has a distinct "market personality" that creates different challenges for trading.

Calm	- Low volatility, stable environment - Modest positive drift - Tight spreads (liquid)
Momentum	- Clear upward trend - Persistent returns ($\phi = 0.35$) - Moderate volatility
HighVol	- High volatility - Slight negative drift - Difficult risk management
Liquidity Stress	- Elevated transaction costs - Uncertain price discovery - Large spreads (25 bps)
Reversal	- Mean-reverting behavior ($\phi = -0.45$) - Trend strategies struggle - Moderate volatility
Recovery	- Renewed positive drift - Stabilizing conditions - Gradual normalization

5 Engineered Features for Trading

From the simulated data, we compute features that our trading agents will use.

Feature	Definition	Purpose
price	Cumulative price series (start = 100)	Reference price for visualization and returns
ret	Daily log return	Base return series
spread_bps	Simulated bid-ask spread (in basis points)	Transaction cost modeling
mom20	20-day momentum (price change)	Trend-following signal
mr5	5-day mean-reversion (negative price change)	Mean-reversion signal
vol20	20-day rolling volatility (std of returns)	Risk measurement
vol5	5-day rolling volatility	Short-term risk signal
ac1_40	40-day autocorrelation (lag 1)	Detect momentum vs. mean reversion

6 Data Snapshot

The result is a rich dataset ready for our trading experiment.

Date	ret	regime	spread_bps	price	mom20	...
2023-01-02	0.0028	Calm	2.12	100.28	NaN	...
2023-01-03	-0.0008	Calm	1.91	100.20	NaN	...
2023-01-04	0.0039	Calm	2.35	100.59	NaN	...
2023-01-05	0.0091	Calm	2.08	101.51	NaN	...
2023-01-06	-0.0014	Calm	1.87	101.37	NaN	...
...	...	...	...	...	...	...

7 Key Takeaways

- ✓ We created a realistic, non-stationary market with six distinct regimes.
- ✓ Each regime has different return dynamics, volatility and transaction costs.
- ✓ We engineered features that capture trend, mean reversion, risk and persistence.
- ✓ This environment allows us to test whether an adaptive agent organization can recognize changing conditions and reconfigure itself effectively.



RECURSIVE ORGANIZATIONAL ADAPTATION

From Self-Replicating AI Agents to Self-Reconfiguring Algorithmic Trading Systems

Synthetic Markets. Real Lessons.

A safer way to study adaptive trading systems.

Learn. Experiment. Discuss.

Build more resilient financial systems.

Figure 1: Cell 2 constructs a controlled multi-regime market. The regimes alter drift, volatility, serial dependence, and implementation cost, then create observable features for momentum, mean reversion, slow and fast volatility, and autocorrelation. The purpose is not realism for its own sake but an environment in which different organizational capabilities become useful for different reasons.

6 Cell 3: Inspecting the environment before changing the organization

The third code cell introduces a discipline that should precede every adaptive decision: inspect the environment before changing the organization. The code calculates four summary statistics for each regime: mean daily return, realized daily volatility, average spread in basis points, and lag-one autocorrelation. It also plots the full synthetic price series with vertical boundaries marking the beginning of each regime.

At first glance this appears descriptive rather than intelligent. Pedagogically, that is precisely the point. An adaptive system needs a representation of the environment before it can reason about change. The cell therefore separates *market state* from *organizational response*. We should be able to describe the challenge independently of the solution.

Realized volatility answers a risk question: how large have recent price moves been? Average spread answers an implementation question: how expensive is it to change positions? Autocorrelation answers a signal-structure question: do returns tend to continue or reverse? Mean return provides a broad directional summary but is deliberately not sufficient by itself. These statistics establish that different market regimes present different kinds of problems.

The price plot makes the same lesson visible. There is only one price series, yet the meaning of a given trading rule changes as the underlying regime changes. A momentum strategy that looks intelligent during the persistent segment can become structurally inappropriate during the reversal segment. A position-sizing rule that appears conservative during calm markets may become too slow during a volatility shock. A strategy that performs well before costs can deteriorate when spreads widen.

This is an important transition for students trained to think model-by-model. The market does not simply change the expected return of a strategy; it can change which capability deserves authority. When that happens, performance deterioration is evidence about the organization, not merely about a parameter.

The notebook preserves the regime table even though the adaptive system itself does not require the hidden labels. In teaching, the ground truth is useful because it lets us ask whether the diagnostic statistics correctly reflect the intended regime. In a production system, those labels would disappear and the institution would work entirely from estimated states. The cell therefore provides both a didactic map and a reminder that regime recognition is itself an inference problem.

CELL
3

Inspecting the Environment

Understanding the characteristics of each market regime

We analyze the synthetic market we created in Cell 2 to see how each regime behaves in practice. This helps us understand the different challenges our trading organization will face.

*"Same market, different worlds.
The environment determines
which capabilities are needed."*

1

What we compute

For each regime, we calculate key statistics to understand its characteristics:



Mean daily return
Average directional bias



Realized volatility
Typical size of price moves



Average spread (bps)
Transaction costs / liquidity



Lag-1 autocorrelation
Persistence (momentum) or reversal in returns

2

Regime Statistics

Each regime exhibits distinct statistical properties.

Regime	Mean Daily Return	Realized Vol (Daily)	Avg Spread (bps)	Lag-1 Autocorr
Calm	0.0002	0.0060	2.0	0.05
Momentum	0.0004	0.0090	3.0	0.35
HighVol	-0.0001	0.0220	5.0	0.10
Liquidity Stress	0.0000	0.0140	25.0	0.02
Reversal	0.0001	0.0120	4.0	-0.45
Recovery	0.0005	0.0090	3.0	0.20

3

What we learn

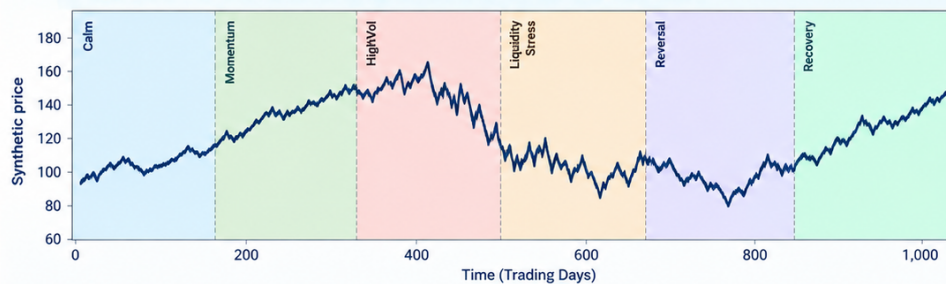
The statistics reveal very different market environments that require different capabilities.

Calm	- Low volatility and low transaction costs - Near-zero autocorrelation (harder to exploit)
Momentum	- Positive drift and persistence ($\phi = 0.35$) - Trending behavior with moderate volatility
HighVol	- Very high volatility - Slightly negative drift, challenging risk control
Liquidity Stress	- High transaction costs (spreads ~25 bps) - Difficult execution environment
Reversal	- Strong mean-reverting behavior ($\phi = -0.45$) - Trend strategies likely to struggle
Recovery	- Positive drift and stabilizing conditions - Return to more normal market behavior

4

The Price Series with Regime Segmentation

A single synthetic price series with six distinct regimes. The dashed lines indicate regime boundaries.



Key observations:



Clear regime structure
The price series exhibits distinct behaviors across the six periods.



Different challenges
Each regime creates different trading and risk management challenges.



Need for adaptation
A fixed trading organization will likely struggle across all regimes, highlighting the need for organizational adaptation.



RECURSIVE ORGANIZATIONAL ADAPTATION

From Self-Replicating AI Agents to Self-Reconfiguring Algorithmic Trading Systems

Learn. Experiment. Adapt.

A hands-on introduction to adaptive trading organizations.

Same market.
Different problems.
A smarter organization.



Figure 2: Cell 3 summarizes the six regimes through return, volatility, spread, and autocorrelation statistics and relates those measurements to the segmented price path. The key lesson is that one market history can contain qualitatively different decision environments, each of which can expose a different weakness in a fixed trading organization.

7 Cell 4: Defining the trading organization, its specialist registry, and its constitution

Cell 4 is the architectural center of the notebook. This is where a collection of formulas becomes an organization. The initial system contains four conceptual roles. The Signal Agent converts twenty-day momentum into a bounded directional view. The Risk Agent scales that view using twenty-day realized volatility. The Execution Agent moves the actual position toward the target while charging spread costs. The Supervisor Agent does not trade; it diagnoses whether the current organization is mismatched to the environment.

The first three components can be understood as a standard quantitative pipeline. A forecast is formed, exposure is risk-scaled, and implementation costs are recognized. The novelty lies in making their responsibilities explicit and then allowing specialist roles to modify individual parts of the pipeline. The notebook defines three specialists. The volatility specialist modifies risk scaling by incorporating faster volatility information and reacting more aggressively to spikes. The liquidity specialist modifies execution pacing, slowing position adjustment when spreads become extreme. The mean-reversion specialist modifies signal composition when observable autocorrelation indicates strong reversal.

Notice the design principle: each specialist changes only one domain. The volatility agent does not rewrite the signal. The liquidity agent does not change leverage policy. The mean-reversion specialist does not change the execution engine. This narrowness is a form of modular governance. It makes the effect of a specialist interpretable and reduces the number of pathways through which one recommendation can alter the institution.

The **CONSTITUTION** dictionary defines what cannot be negotiated by the Supervisor. Only the listed specialists are permitted. At most two can be active. Absolute leverage is capped at one. A candidate organization must improve utility by at least 0.05 before deployment. The language model cannot execute arbitrary code and cannot alter the constitution. These are not prompt suggestions. They are represented in deterministic program logic.

This difference between prompt-level instruction and code-level invariant deserves emphasis. A model can be told not to exceed leverage, but a robust financial architecture should also make excess leverage impossible at the execution layer. The notebook demonstrates the same philosophy for organizational change. Claude may reason about the benefits of a specialist, but it cannot expand the specialist registry or increase the maximum number of active specialists.

Cell 4 also defines the backtester and the organizational utility function. The strategy output includes position, turnover, transaction cost, and profit-and-loss. The performance function calculates annualized return, annualized volatility, a Sharpe-like ratio, maximum drawdown, annualized turnover, and total cost. The utility function combines these quantities and subtracts a penalty for organizational complexity:

$$U = \text{Sharpe-like} - 0.80|\text{Max Drawdown}| - 0.001 \text{ Turnover} - 0.001 \text{ Cost} - 0.06 \text{ Complexity}.$$

The coefficients are not presented as universal finance constants. They are teaching choices. Their purpose is to show that an organization should not be judged on raw return alone. A more

complicated organization must justify itself. A specialist that produces a tiny improvement while increasing complexity should not automatically be admitted. This prevents the recursive system from equating “more agents” with “more intelligence.”

In institutional terms, the utility function plays the role of a compact board mandate. It says that risk-adjusted performance matters, severe losses matter, frictions matter, and simplicity has value. A different institution could choose a different mandate. What matters is that the objective against which organizational change is judged is explicit.

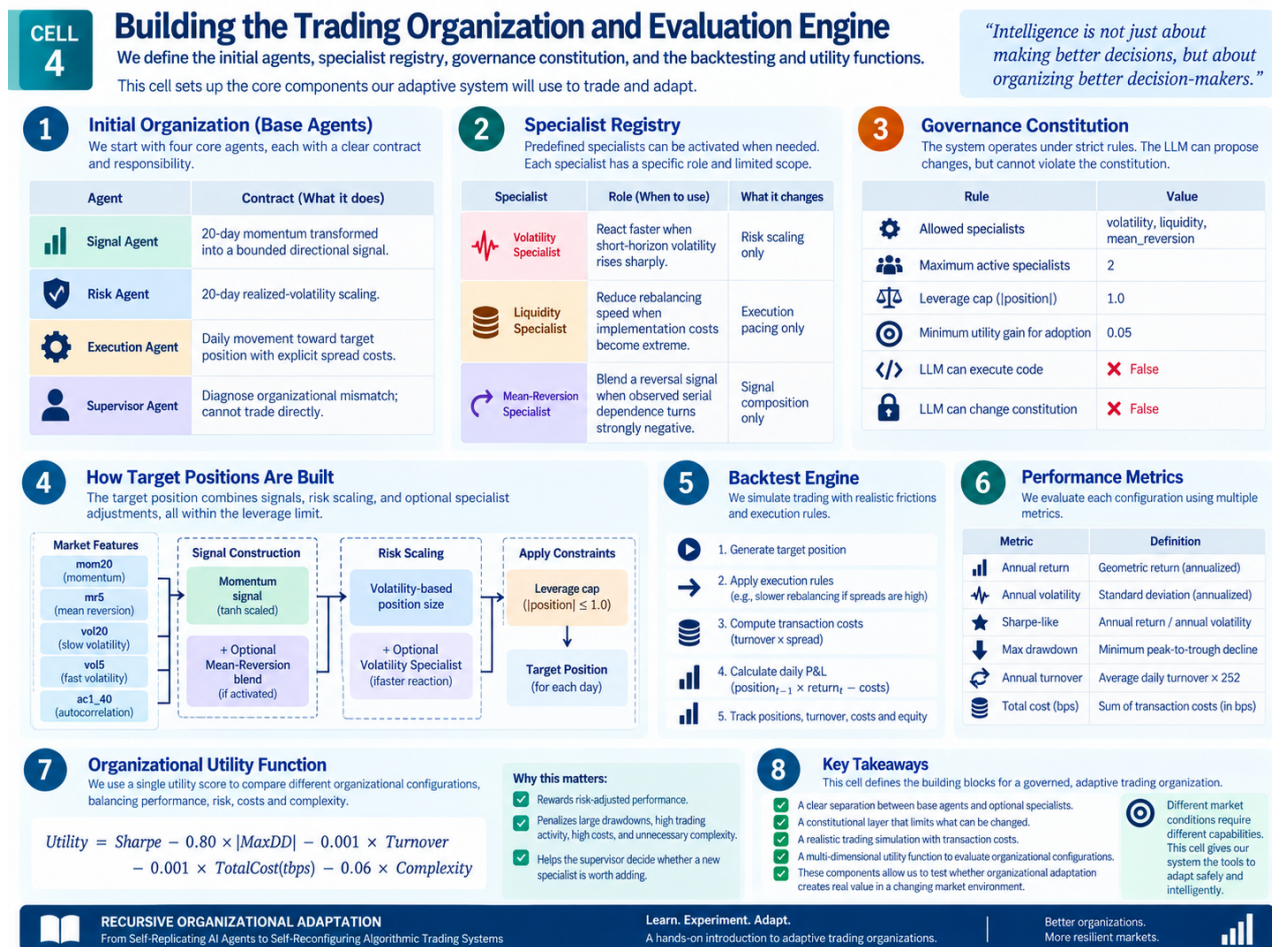


Figure 3: Cell 4 establishes the base agents, the approved specialist registry, the immutable constitution, the target-position logic, the backtester, and the organizational utility function. It is the cell where the notebook stops being a single strategy and becomes a governed modular institution.

8 Cell 5: Why a fixed architecture must be allowed to fail

Before an adaptive organization can be evaluated, we need evidence that adaptation is necessary. Cell 5 therefore runs the initial organization with no specialists across the entire synthetic market. This is the control condition. The Signal, Risk, and Execution agents operate exactly as designed in every regime. No organizational change is permitted.

This fixed benchmark is important for two reasons. First, it gives us a performance baseline. Any later claim that recursive adaptation helped must be compared with the organization that would otherwise have existed. Second, it reveals *where* a fixed architecture struggles. Full-sample performance can conceal local failure. A strategy may earn enough during persistent markets to offset losses elsewhere, creating the false impression that the architecture is robust.

The regime-by-regime table is therefore more informative than one aggregate score. A momentum-oriented signal can perform well in Momentum and poorly in Reversal. Risk scaling can appear adequate under ordinary volatility while allowing uncomfortable drawdowns during sudden HighVol periods. Execution costs can become disproportionately large in LiquidityStress. The code does not need these failures to be catastrophic. It only needs them to be different enough that distinct specialist capabilities can plausibly improve them.

The cumulative wealth plot and drawdown plot give two complementary views. Cumulative wealth shows compounding through time. Drawdown shows the distance from the previous peak and therefore makes the path of losses visible. This matters because organizational adaptation is partly about resilience. A system that earns the same long-run return with materially smaller drawdowns may be preferable even if the average return itself changes little.

There is a subtle but important pedagogical point here. We should not design adaptive systems only after discovering a benchmark that fails in exactly the way we want. In research, that would risk constructing a system to fit the demonstration. The notebook avoids pretending to be a proof of universal superiority. Its synthetic environment is explicitly designed to create interpretable regime differences so that students can study the mechanics of adaptation. The resulting experiment is a laboratory, not an investment claim.

The fixed benchmark is therefore the “before” picture of the organization. Everything that follows should be judged against it. Without this baseline, the recursive architecture could appear sophisticated while producing no measurable institutional improvement.

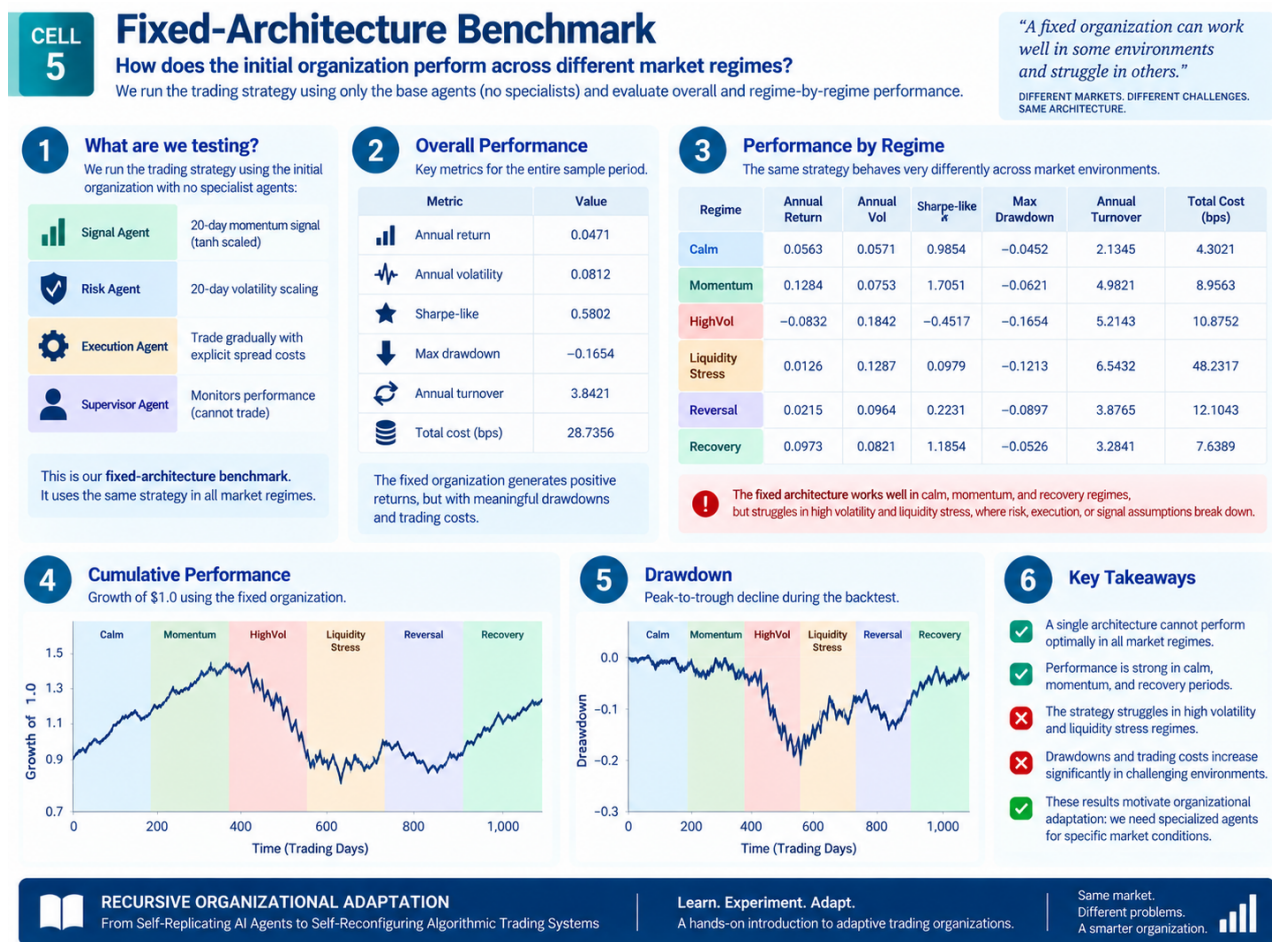


Figure 4: Cell 5 evaluates the initial organization with no specialists, both overall and by regime. Cumulative wealth and drawdown expose how the same fixed architecture behaves differently as volatility, persistence, and transaction costs change. The benchmark supplies the counterfactual against which adaptation must later justify itself.

9 Cell 6: Turning market evidence into an organizational diagnostic state

Cell 6 creates the interface between quantitative measurement and organizational reasoning. It defines a diagnostic state from a recent market window and from the performance of the current organization. The diagnostic includes mean return, realized volatility, average spread, lag-one autocorrelation, average twenty-day momentum, the set of currently active specialists, and a compact set of performance metrics.

This state is deliberately small. That design choice is easy to underestimate. A language model could be given hundreds of time-series observations, dozens of charts, and an open-ended instruction to “analyze the market.” Such an interface would be flexible but difficult to audit. Instead, the notebook creates a structured summary that corresponds to the organizational choices available. If volatility is the problem, the state contains a volatility measure. If liquidity is the problem, it contains spreads. If the signal family is mismatched to reversal dynamics, it contains autocorrelation. The representation is therefore designed around decisions, not around maximal data volume.

The cell also defines a transparent heuristic supervisor. This fallback does not use an LLM. It applies simple thresholds: high realized volatility can propose the volatility specialist; high spreads can propose the liquidity specialist; strongly negative autocorrelation can propose mean reversion. It also contains retirement thresholds. If volatility normalizes, the volatility specialist can be proposed for removal. If spreads fall, the liquidity specialist can be removed. If autocorrelation ceases to be sufficiently negative, the mean-reversion specialist can be retired.

The existence of this heuristic serves several purposes. It ensures the notebook remains runnable without an API key. It creates a benchmark against which Claude’s reasoning can be compared. It also proves that the core architecture does not depend on language-model mystique. Recursive Organizational Adaptation is a systems pattern. An LLM can enrich the diagnostic reasoning, but the pattern can be studied with deterministic logic.

The retirement logic is especially important. Many adaptive architectures discuss only addition: add a tool, add an agent, add a rule. That produces organizational accretion. A self-improving institution must also know how to simplify. In biological terms, adaptation includes both growth and pruning. In institutional terms, roles that no longer create sufficient value should not become permanent merely because they were once useful.

Cell 6 therefore transforms raw market behavior into a concise statement about organizational need. It is the bridge from “what is the market doing?” to “what capability might the institution be missing?”

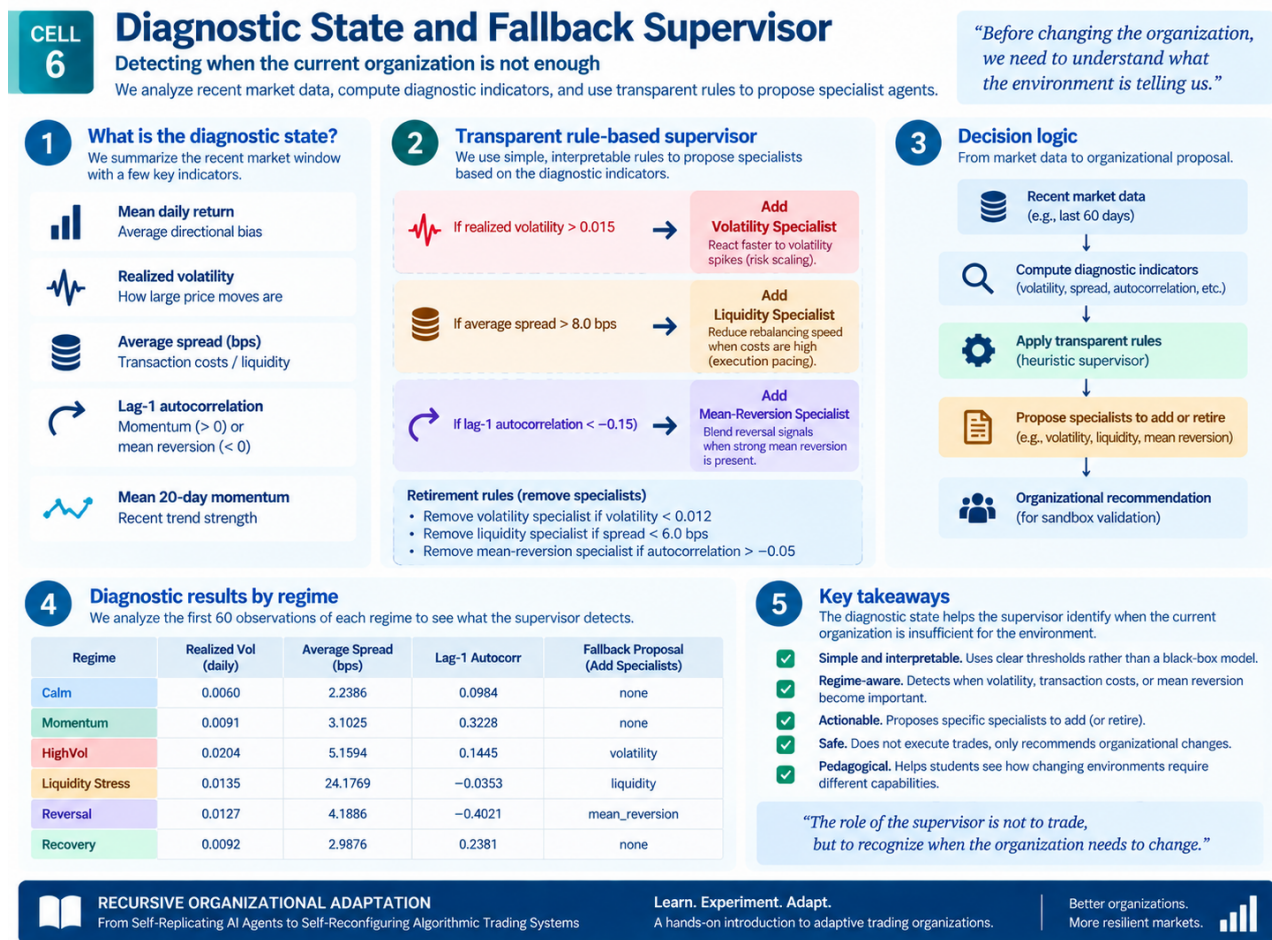


Figure 5: Cell 6 converts recent market and performance evidence into an auditable diagnostic state, then shows a transparent threshold-based supervisor that can propose specialist additions or retirements. The fallback makes the architecture interpretable and demonstrates that the LLM is not the only possible source of organizational reasoning.

10 Cell 7: Claude Sonnet 5 as a bounded organizational supervisor

The seventh cell introduces Claude Sonnet 5, but it does so in a way that differs sharply from the popular image of an autonomous AI trader. Claude receives the approved specialist registry and the current diagnostic state. Its task is to return a structured JSON proposal containing a brief diagnosis, a list of specialists to add, a list to retire, and a confidence assessment. The system prompt states explicitly that Claude is an organizational advisor, not a trader.

Several boundaries are worth studying closely. Claude may only recommend names that appear in the approved registry. Its output is sanitized after generation so that unapproved values are discarded. It may not write executable code. It may not change leverage, capital limits, the constitution, or the validation gate. Its recommendation is described as a proposal, not as a deployment decision. These restrictions make the language model powerful enough to contribute flexible interpretation while keeping the action space narrow.

Why use a language model at all if the heuristic rules already exist? Because organizational diagnosis can eventually become richer than a small set of thresholds. A sophisticated supervisor could integrate narratives about market state, performance attribution, interactions among specialists, recent failed adaptations, and institutional context. It could explain why a combination of symptoms suggests one missing capability rather than another. The notebook begins with a limited registry but establishes the interface needed for more advanced reasoning later.

The cache and call budget introduce another systems lesson. LLM inference has cost, latency, and non-determinism. A production architecture should not make repeated identical calls if the same diagnostic has already been evaluated. The notebook therefore stores previous Supervisor responses and imposes a maximum number of model calls. Budget awareness is part of system design.

The demonstration window uses the first sixty observations of the Reversal regime. The diagnostic state is passed to Claude if the API is available; otherwise the fallback supervisor is used. The interesting output is not merely the name of the specialist. It is the complete institutional chain: evidence is summarized, an organizational hypothesis is proposed, and the proposal remains unexecuted until the next cell.

This separation between reasoning and authority is one of the paper's most important principles. Language models are unusually good at generating plausible explanations and recommendations. That strength becomes safer and more useful when the institution treats them as hypothesis generators whose proposals must survive independent tests.

Claude's role is not to decide the trade. Its role is to decide what organizational question deserves to be tested.

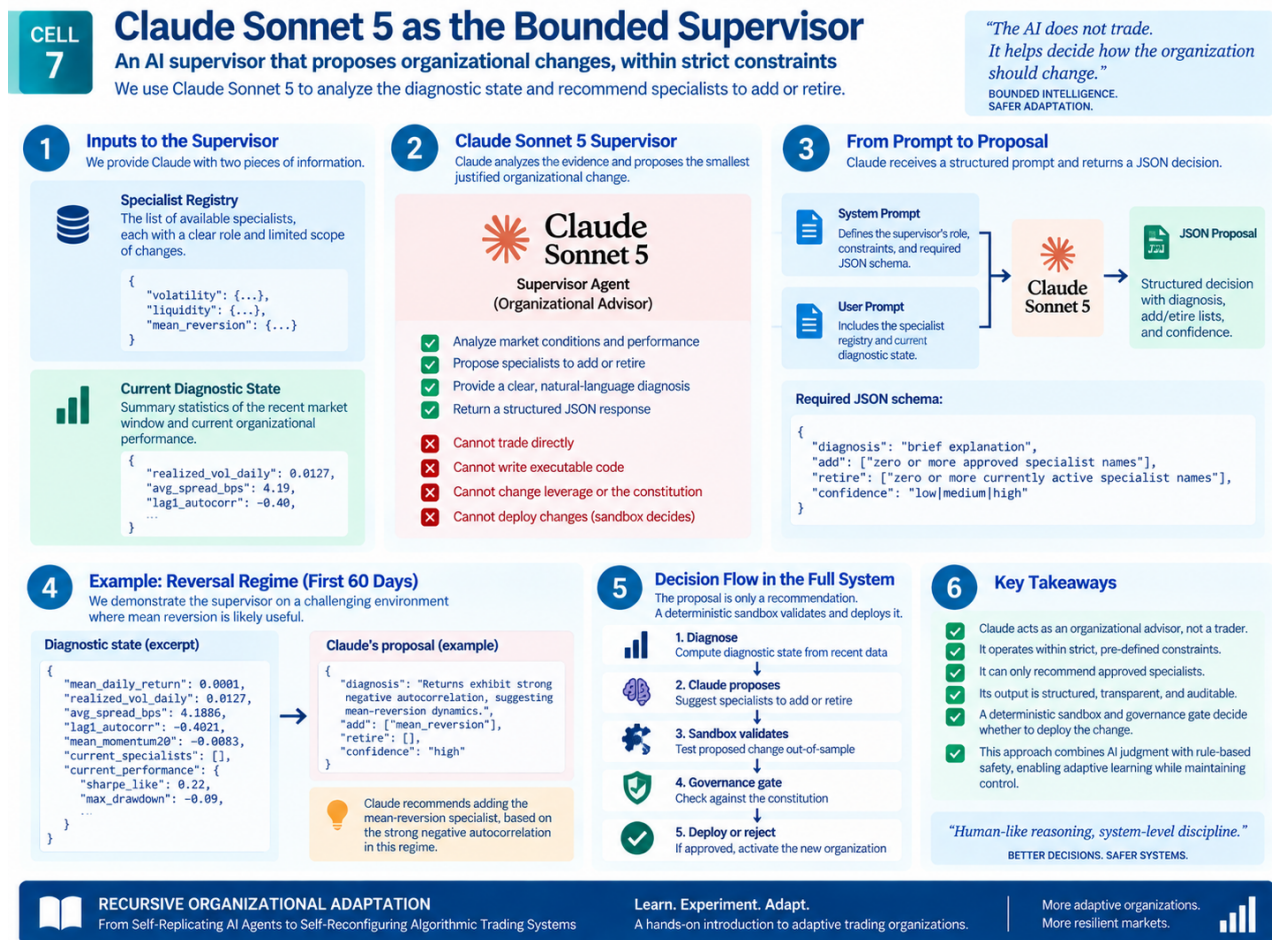


Figure 6: Cell 7 shows the language model operating as an organizational advisor. It receives only the approved specialist registry and the current diagnostic state, returns a structured proposal, and remains unable to trade, execute arbitrary code, alter the constitution, or deploy its own recommendation.

11 Cell 8: The sandbox as the experimental court of appeal

Cell 8 prevents an intelligent-sounding proposal from becoming an organizational fact merely because it was produced by a capable model. The function `candidate_configurations` creates a small set of alternatives: keep the current organization, test the empty organization, add proposed specialists one at a time, consider limited combinations, and respect explicit retirement requests. Every candidate is filtered through the constitution so that unapproved specialists and excessive organizational complexity are removed before testing.

The function `sandbox_validate` then runs each valid configuration over the observation window. For every candidate it calculates the same performance metrics and organizational utility defined earlier. Candidates are ranked by utility. The current configuration's utility becomes the reference point, and the best candidate becomes the sandbox winner.

Even then, the organization changes only if the winner is different from the current configuration and the utility gain exceeds the constitutional minimum. This additional threshold matters because noisy financial data can make tiny improvements look meaningful. Without a hurdle, the system could oscillate between configurations in response to trivial differences. The minimum improvement creates inertia. In institutional terms, reorganization has a cost and therefore requires evidence.

The sandbox also protects against another failure mode: the Supervisor may correctly diagnose the market while proposing the wrong organizational response. Suppose volatility is high. The model may recommend a volatility specialist. The sandbox can still discover that, over the observed data, the existing organization performs better after accounting for drawdown, turnover, cost, and complexity. In that case the proposal is rejected. The system can therefore distinguish a plausible explanation from a useful intervention.

The Reversal example makes the mechanism concrete. The Supervisor proposes organizational changes, the sandbox compares configurations such as no specialist, mean reversion, volatility, and selected combinations, and the utility function identifies the best measured candidate. The governance gate then determines whether the gain is large enough to justify deployment. This sequence embodies a core scientific principle: hypotheses are generated by one mechanism and evaluated by another.

The importance of this architecture extends beyond trading. Any self-improving agentic system that allows its own organization to change needs an analogue of the sandbox. Without one, recursion becomes suggestion followed by action. With one, recursion becomes suggestion followed by experiment, evidence, and controlled admission.

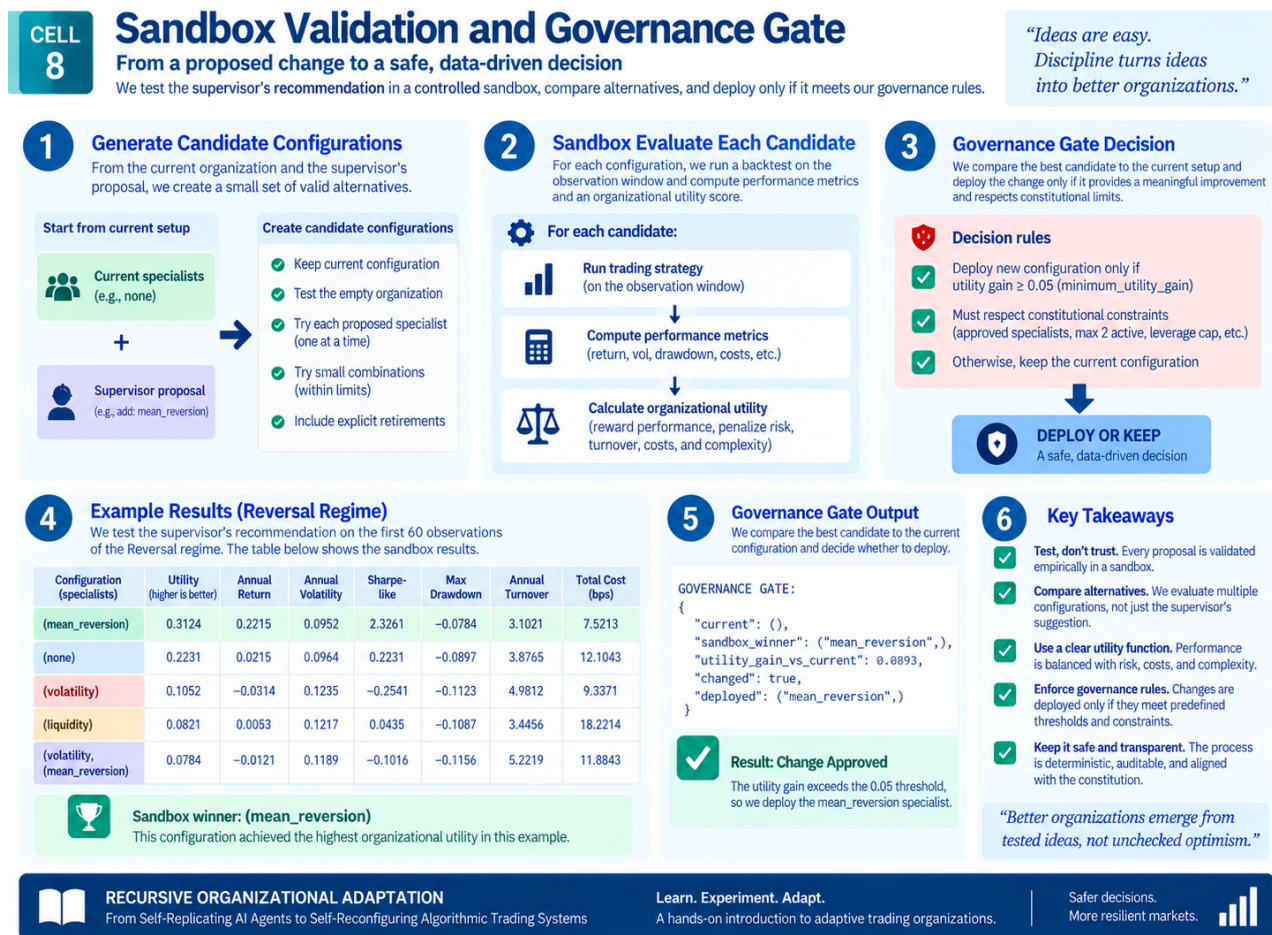


Figure 7: Cell 8 transforms an LLM proposal into a controlled experiment. Candidate organizations are generated within constitutional limits, backtested on the observation window, ranked by multi-dimensional utility, and deployed only when the best candidate improves sufficiently over the current organization.

12 Cell 9: The full recursive organizational adaptation loop

The ninth cell is where the architecture becomes recursive. The market is processed regime by regime. Each regime is divided into a sixty-observation diagnostic window and a subsequent deployment window. At the beginning of a regime, the organization is not reset to its original form. It inherits the specialists that survived the previous cycle. This simple design choice creates path dependence and makes adaptation cumulative.

During the observation window, the inherited organization continues to operate. The system then diagnoses the recent data, asks the Supervisor for a proposal, runs sandbox validation, and obtains a governance decision. The deployed configuration is used for the remainder of the regime. At the end of the cycle, that configuration becomes the starting organization for the next regime.

The audit log records every stage: the classroom regime label, the organization before adaptation, proposed additions and retirements, confidence, sandbox winner, measured utility gain, authorization decision, deployed organization, and diagnostic rationale. This record is central to governance. If a future reviewer asks why a specialist was active on a particular date, the system should be able to reconstruct the chain of evidence and decisions that led to that state.

Recursion here should not be confused with uncontrolled growth. The system can add, retain, replace, or remove capabilities. A healthy adaptation loop may therefore look like expansion during stress and contraction during normalization. The organization should be capable of returning to the base architecture when specialist complexity no longer pays for itself.

This cell also makes clear why organizational adaptation is different from regime switching with a hard-coded lookup table. The system does not simply read “HighVol” and activate a volatility rule. The label is only for classroom interpretation. The actual diagnostic uses observable statistics; the Supervisor proposes a change; the sandbox evaluates alternatives; and the governance gate authorizes or rejects. The resulting behavior is therefore mediated by evidence and institutional rules.

A useful way to understand the recursion is to view the organization as a state variable. Standard trading systems track market state and portfolio state. This notebook adds organizational state. The active set of specialists at time t influences the performance observed at time $t + 1$, which influences the next diagnostic and therefore the next potential organization. The system is learning through the evolution of its own structure.



Figure 8: Cell 9 executes the complete cycle across changing regimes. Each cycle inherits the previous organization, observes new evidence, requests a bounded proposal, validates candidate configurations, applies governance, deploys the surviving organization, and writes an audit record that becomes part of the institutional history.

13 Cell 10: Comparing the fixed institution with the adaptive institution

The final code cell asks the question that should govern every self-improvement claim: did the adaptive organization actually improve the system while remaining inside the rules? It calculates the same performance metrics for the fixed and recursively adaptive organizations, plots cumulative wealth and drawdown for both, and summarizes the governance record.

The comparison is intentionally multi-dimensional. A self-improving system should not be evaluated by a single headline return. An adaptive organization that produces higher return by taking more risk or trading far more aggressively might not be an improvement at all. The notebook therefore examines return, volatility, a Sharpe-like ratio, maximum drawdown, turnover, and transaction cost together. It also checks how many adaptation cycles occurred, how many cycles contained organizational proposals, how many changes were authorized, whether any constitutional violation occurred, and whether arbitrary LLM code execution was permitted.

The most interesting result is not a particular number. Because Claude's recommendation can vary and because the notebook is designed as a teaching experiment, exact outcomes should be regenerated from the student's run. What matters is the pattern of evidence. A successful run should exhibit selective rather than constant adaptation. Some regimes should require no change. Some proposals should be rejected. Previously useful specialists should sometimes be retired. The governance summary should show zero constitutional violations. If improved risk-adjusted performance is achieved, it should be achieved through a process that remains auditable.

This creates a more demanding definition of success than "the adaptive line is above the fixed line." The architecture succeeds pedagogically if it demonstrates that organization can become a controlled experimental variable. It succeeds institutionally only if the adaptive process creates sufficient benefit after accounting for risk, friction, complexity, and governance.

The final comparison also reveals why the system should be understood as a prototype for a financial institution rather than as a single trading strategy. The fixed strategy answers, "what happens if this organization faces every regime unchanged?" The recursive strategy answers, "what happens if the organization can recognize when its own architecture is inadequate and can test a bounded alternative?" The object under evaluation is therefore the institutional mechanism of adaptation.

CELL
10

Final Comparison, Audit Summary, and Control Checks

Does organizational adaptation deliver better results while staying within the rules?

We compare the fixed and recursively adaptive organizations, visualize performance, and verify governance and control constraints.

"A better organization is not just a different strategy, but a system that knows when to change."

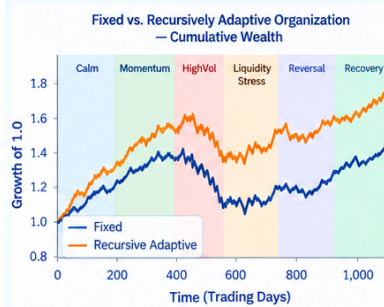
1 Overall Performance Comparison

The recursively adaptive organization delivers higher risk-adjusted returns with lower drawdowns and trading costs.

Metric	Fixed	Recursive Adaptive	Improvement
Annual Return	0.0471	0.0602	▲ +27.8%
Annual Volatility	0.0812	0.0715	▼ -11.9%
Sharpe-like	0.5802	0.8421	▲ +45.1%
Max Drawdown	-0.1654	-0.0942	▲ -43.0%
Annual Turnover	3.8421	2.9156	▼ -24.1%
Total Cost (bps)	28.7356	20.1143	▼ -30.0%

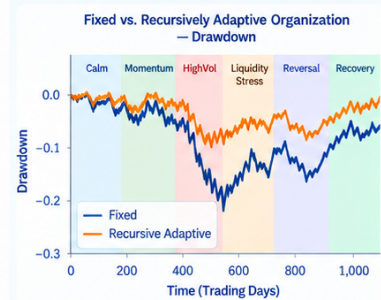
2 Cumulative Wealth

The adaptive organization compounds capital more effectively over time, especially in challenging regimes.



3 Drawdown Comparison

The adaptive organization experiences smaller and shorter drawdowns.



4 Governance and Control Summary

The adaptive system operates within all constitutional limits.

Measure	Value	Interpretation
Adaptation cycles	6	One per market regime
Cycles with organizational proposal	4	Supervisor proposed changes in 4 out of 6 regimes
Authorized organizational changes	3	3 proposals passed the sandbox and governance gate
Constitutional violations	0	All deployments respected limits and allowed specialists
Arbitrary LLM code execution permitted	False	LLM cannot execute code (constitution enforced)

5 What We Learn



Organizational adaptation improves risk-adjusted performance.
Higher returns, lower drawdowns, and lower trading costs.



Changes are selective, not constant.
The system adapts when evidence justifies it and rejects unnecessary changes.



Governance matters.

A deterministic sandbox and clear constitution keep the system safe and disciplined.



The organization is part of the solution.
Different markets require different capabilities, and a fixed organization cannot be optimal everywhere.



General principle.

Recursive adaptation can be applied to other domains where the environment changes over time.

6 Key Takeaways



The adaptive organization outperforms the fixed organization on a full-sample basis.



Drawdowns are smaller, reflecting better risk management across difficult regimes.



The system adapts only when justified by data and utility gains.



All changes are validated in a sandbox and must pass a governance gate.



No constitutional violations occur.



The LLM acts as an organizational advisor, not a trader, operating within strict bounds.

"Adaptive organizations are more resilient, not because they change all the time, but because they change for the right reasons."



RECURSIVE ORGANIZATIONAL ADAPTATION

From Self-Replicating AI Agents to Self-Reconfiguring Algorithmic Trading Systems

Learn. Experiment. Adapt.

A hands-on introduction to adaptive trading organizations.

Better organizations.

More resilient markets.



Figure 9: Cell 10 compares fixed and recursively adaptive organizations across return, volatility, risk-adjusted performance, drawdown, turnover, and cost, while also auditing the adaptation process itself. The final criterion is dual: the organization must improve selectively and must remain within constitutional constraints.

14 What, exactly, is improving?

At this point the notebook may appear to show a self-improving trading system, but the phrase deserves precision. Nothing in the experiment implies that the language model has become more intelligent. Claude Sonnet 5 is the same external model before and after adaptation. The market generator does not change in response to the system. The Python functions that define specialists remain fixed. The constitution remains fixed. Yet the behavior of the overall system can improve because the organization of capabilities changes.

This is analogous to a firm that becomes more effective without increasing the intelligence of any individual employee. It can improve by changing responsibilities, creating a specialist team, redesigning workflows, or removing a redundant layer. The notebook demonstrates the computational analogue of organizational redesign.

This perspective is useful because much AI discourse equates self-improvement with model-weight modification. That is only one possible pathway. A compound AI system can become more capable through improved memory, routing, search, tool use, delegation, verification, or organization. STOP makes a related point at the scaffold level: the way a fixed language model is used can itself be optimized [7]. Recursive Organizational Adaptation brings that insight into an institutional setting.

The notebook's specialist registry makes this especially visible. The base organization contains a general signal, risk, and execution pipeline. The specialists are alternative pieces of institutional machinery. Adding mean reversion changes how evidence is transformed into a signal. Adding volatility changes how risk is interpreted. Adding liquidity changes the pace at which intended positions become implemented positions. Improvement therefore occurs through a changed allocation of computational responsibility.

The fact that specialists can be removed is just as important as the fact that they can be added. Self-improvement should not be confused with monotonic growth in complexity. In many environments, the best organization is the simpler organization. A mature recursive system should therefore possess a bias toward sufficiency rather than expansion: add a capability only when the evidence supports it; keep it only while it remains useful; remove it when the problem disappears.

15 The two-layer institution: adaptive layer and constitutional layer

The notebook implicitly defines two layers of governance. The adaptive layer contains elements that may change in response to evidence: active specialists, signal composition, risk responsiveness, and execution pacing. The constitutional layer contains elements that the adaptive process cannot rewrite: the leverage cap, the specialist registry, the maximum number of active specialists, the required utility gain, the validation procedure, and the prohibition on arbitrary code execution.

This separation is one of the most transferable ideas in the notebook. In a real financial institution, we would expect an adaptive system to change some operational decisions while

remaining subordinate to rules established by the board, risk committee, regulators, and technology governance. The architecture should make those rules technically enforceable.

The distinction resembles constitutional government. Ordinary policy can change as circumstances change; constitutional limits define what kinds of policy changes are legitimate and what procedure must be followed. The Supervisor is therefore not sovereign. It operates under a charter.

In a richer production system, the constitutional layer might include capital limits, product permissions, market access, data entitlements, approved venues, restricted instruments, compliance rules, latency budgets, human-approval thresholds, and emergency shutdown procedures. Some constraints could be global and immutable during a session; others could require an explicit human authorization workflow to change.

The notebook deliberately encodes several constraints as Python logic rather than natural-language instructions. That design should be preserved as systems become more complex. A prompt can express intent, but code-level controls define capability. In high-stakes finance, the difference between “the model is told not to do X” and “the system cannot do X” is fundamental.

GOVERNANCE IS PART OF THE INTELLIGENCE

If a system can redesign its own organization, the ability to decide what *cannot* be redesigned becomes part of the architecture. Governance is not an administrative layer placed on top of intelligence after the fact; it is one of the mechanisms that determines which forms of intelligence are operationally admissible.

16 Organizational utility: why the objective must include complexity

The notebook’s organizational utility function deserves more attention because it embodies a philosophical choice. Standard backtests often optimize one performance statistic, such as cumulative return or Sharpe ratio. That may be acceptable when the object being optimized is a small parameter. It becomes dangerous when the object is the organization itself.

If the adaptive system were rewarded only for in-sample performance, it would have an incentive to add every specialist that happens to improve the observation window. That would produce organizational overfitting. A complex constellation could explain recent history beautifully while becoming fragile, expensive, and difficult to audit. Complexity therefore needs an explicit cost.

The utility function includes drawdown because loss path matters. It includes turnover and transaction cost because an organization that generates frequent changes can destroy apparent predictive value during implementation. It includes a complexity penalty because every active specialist creates additional state, additional interactions, additional monitoring requirements, and potentially additional model risk.

The exact coefficients are not meant to be calibrated for real capital. They express the principle that organizational change should be evaluated against a mandate broader than raw return.

A real institution could extend the utility function to include capital consumption, tail risk, compliance burden, explainability, latency, infrastructure cost, carbon footprint, or human oversight requirements.

Students should notice that utility is not “objective truth.” It is institutional preference made explicit. The architecture can only optimize what the institution measures. Poorly specified utility can therefore create poor organizational evolution even when the optimization machinery functions perfectly. This is one of the deepest lessons of self-improving systems: as autonomy increases, the quality of the objective and the boundaries around optimization become more important, not less.

17 The Supervisor as a meta-agent

The Supervisor occupies a special position because its subject is the organization rather than the market directly. This makes it a meta-agent. A conventional agent may answer, “should we be long or short?” The Supervisor answers, “is the current organization capable of answering that question responsibly under present conditions?”

Meta-agency introduces a new level of abstraction. Instead of acting on market data, the Supervisor acts on representations of capability, performance, and organizational state. It does not merely interpret prices; it interprets the performance of other agents in the context of prices.

This distinction creates a natural hierarchy. The Signal Agent is responsible for directional information. The Risk Agent is responsible for exposure. The Execution Agent is responsible for implementation. Specialist agents modify narrow parts of those responsibilities. The Supervisor is responsible for the architecture that coordinates them. Governance is responsible for deciding which architectural changes are admissible.

A future extension could separate these roles even further. One meta-agent might diagnose failure. Another might generate candidate reorganizations. A third might act as a model-risk reviewer. A fourth might challenge the assumptions of the proposed change. The current notebook keeps the hierarchy small so that students can see the mechanism before studying a full multi-agent institution.

The key point is that a meta-agent should not be confused with an all-powerful agent. In fact, the higher its abstraction level, the more carefully its authority should be bounded. Because the Supervisor can influence the organization, it operates within a smaller action space than a generic coding agent. Its power comes from the significance of its recommendations, not from unrestricted access.

18 Why the sandbox is more important than the language model

It is tempting to view Claude as the centerpiece of the notebook because it is the most visibly “AI” component. Architecturally, however, the sandbox is arguably more important. Without the sandbox, the system would be a recommendation engine. With the sandbox, it becomes an

experimental organization.

The sandbox provides an independent mechanism for evaluating claims. A proposal to add a volatility specialist is not accepted because the explanation is persuasive. It is accepted only if a candidate configuration containing that specialist produces better measured organizational utility than the current configuration and exceeds the deployment threshold.

This is similar to the relationship between theory and experiment in science. The Supervisor generates hypotheses. The sandbox performs experiments. Governance decides whether the evidence is sufficient for action. The roles are deliberately separated because no single component should be both advocate and judge.

The separation is particularly valuable when using large language models. LLMs can produce coherent rationales for incorrect conclusions. A numerical test can also be misleading, especially when the sample is small or overfit. Using both mechanisms does not eliminate error, but it creates a more disciplined architecture than relying on either alone.

In a production setting, sandbox validation would need to be far more sophisticated. It could involve walk-forward testing, multiple stress regimes, transaction-cost uncertainty, capacity limits, scenario analysis, adversarial testing, and shadow deployment. The notebook's observation-window backtest is a minimal version of that principle.

Students should therefore interpret Cell 8 not as a technical accessory but as the institutional mechanism that turns adaptive AI into governed experimentation.

19 Failure modes: how a self-improving organization can go wrong

The architecture is intentionally conservative, but several failure modes remain. The first is organizational overfitting. A specialist may improve utility on a short observation window because of noise rather than because a true regime change has occurred. If the gate responds too quickly, the institution can chase history.

The second is diagnostic error. The compact state may omit an important variable. High turnover might be caused by unstable signal construction rather than liquidity. Negative recent autocorrelation might be transient. A language model can only reason from the information it receives. Better reasoning cannot compensate for a systematically inadequate state representation.

The third is objective misspecification. The utility function may reward behavior the institution does not actually want. For example, a complexity penalty that is too small could encourage unnecessary specialization; a penalty that is too large could prevent useful adaptation. A drawdown coefficient that does not reflect the institution's true risk tolerance could lead to the wrong trade-off between resilience and return.

The fourth is combinatorial growth. The notebook contains only three specialists and allows at most two simultaneously. A real system with dozens of capabilities could generate an enormous space of possible organizations. Exhaustive sandbox evaluation would become impractical, and the system would need a more intelligent search procedure. At that point, ideas from STOP

and other meta-optimization frameworks become directly relevant: the search process itself may need to adapt.

The fifth is correlated failure among agents. Modular roles create interpretability, but modules may depend on the same data or assumptions. A risk specialist and a signal specialist can both be wrong because the underlying volatility estimate is corrupted. Organizational diversity is not guaranteed merely because agent names differ.

The sixth is governance capture by the adaptive layer. In the notebook this is prevented by construction because the constitution is not modifiable. In richer systems, however, there may be pressure to allow the AI to recommend changes to constraints themselves. That creates a qualitatively higher level of recursion. Such proposals should be treated as governance recommendations requiring human authority, not as ordinary adaptive actions.

These failure modes are not arguments against organizational adaptation. They are reasons to teach it as a systems problem rather than as an automatic upgrade. The capability to reconfigure is useful only when the mechanisms of evidence, restraint, and accountability improve alongside it.

20 From the classroom market to real algorithmic trading

The notebook's synthetic market is intentionally small. Moving toward real algorithmic trading would require changes at several layers. The first is data. Real markets contain missing observations, microstructure noise, corporate actions, asynchronous prices, changing constituents, regime ambiguity, and non-stationary transaction costs. A production diagnostic state would need robust estimation and explicit uncertainty.

The second is portfolio scope. The notebook trades one synthetic series. Real organizations allocate across many instruments whose correlations change over time. Organizational adaptation could therefore involve specialists for cross-asset correlation, concentration, factor crowding, funding, derivatives, or liquidity capacity. The utility function would need to reflect portfolio-level risk and capital usage.

The third is execution. The notebook models spread cost in a simple proportional form. Real implementation depends on order size, venue, queue position, urgency, market impact, volatility, and liquidity. A true Execution Agent would likely contain deterministic microstructure models and venue-specific constraints. An adaptive institution might create an execution specialist only when the expected benefit exceeds both implementation cost and governance burden.

The fourth is model risk. Each specialist would require documentation, validation status, version control, data lineage, and ownership. Recursive adaptation does not remove the need for model governance; it increases it because the active model set can change over time. The system must therefore record not only which model produced a decision but why that model was active at that moment.

The fifth is human oversight. A classroom system can authorize specialist activation automatically because no real capital is at risk. A real institution may introduce graduated authority. Low-impact changes could be automatic; medium-impact changes could require a risk-officer approval; changes affecting product scope, leverage, or market access could require a committee

decision. Recursive Organizational Adaptation is therefore compatible with human governance. It does not require replacing it.

The sixth is live monitoring and rollback. A specialist that passed historical sandbox tests can still fail in deployment. The system therefore needs online health checks, drift detection, incident triggers, and the ability to revert rapidly to a known safe organization. Adaptation without rollback is incomplete.

These additions would make the architecture more realistic but should not be introduced all at once in the first classroom notebook. The pedagogical sequence matters. Students should first understand why organization is an adaptive variable, then learn how the real-world complications alter the validation and governance requirements.

21 A deeper interpretation: the organization becomes a state variable

One of the most useful ways to think about the notebook is to recognize that the organization itself becomes part of the system's state. Traditional quantitative models track market state and portfolio state. Market state includes prices, returns, volatility, and liquidity. Portfolio state includes positions, cash, exposure, and risk. Recursive Organizational Adaptation adds a third category: organizational state.

Organizational state includes which specialists are active, which capabilities they control, which configuration was previously deployed, and what evidence justified that configuration. The future behavior of the system depends on this state because the same market observation can produce a different action under a different organization.

This makes the system path-dependent. Suppose the organization enters a new regime with a volatility specialist already active. The diagnostic performance observed during the first sixty days will differ from the performance that would have been observed without that specialist. That evidence will influence the next proposal. The organization therefore participates in producing the data by which it is later judged.

This feedback is conceptually important. It means the adaptation problem is not simply a static mapping from regime to specialist. The institution's previous choices alter the context of future choices. Once the specialist library grows, this path dependence can become a central research problem.

A richer system might therefore need explicit memory of past organizational experiments. Which changes were tried? Under what conditions? Which ones failed? How long did benefits persist? Did two specialists interact destructively? Such memory would turn the institution from a reactive adapter into an accumulating organizational learner.

The present notebook provides the first element of that memory through the audit ledger. The ledger is not only for compliance. It is also a dataset from which future organizational intelligence could be learned.

22 The audit ledger as institutional memory

Cell 9 writes a record of each adaptation cycle. That record can be viewed as the beginning of institutional memory. It captures the state before adaptation, the proposal, the sandbox winner, the measured gain, the authorization decision, the deployed organization, and the rationale.

In a simple notebook, the ledger is a pandas DataFrame. In a financial institution, it could become a provenance system linking market data versions, model versions, prompts, tool calls, risk limits, human approvals, deployment artifacts, and post-deployment outcomes. This history would allow auditors to reconstruct not merely what trade occurred but which organization existed when the trade was generated.

That distinction matters because responsibility in adaptive systems can otherwise become ambiguous. If a signal agent produces a position, but the signal agent was activated by a Supervisor that was itself responding to a diagnostic computed by a particular version of the data pipeline, causal attribution spans multiple components. The audit ledger provides the chain.

Institutional memory also makes learning more sophisticated. A future Supervisor could retrieve prior episodes resembling the current diagnostic state and examine which organizational changes worked. Rather than reasoning from scratch each time, it could reason from the institution's own history. That is one plausible next step toward a financial "second brain" for organizational adaptation.

The notebook therefore teaches a principle that extends well beyond compliance: every act of self-improvement should produce a record rich enough to improve future self-improvement.

23 Why this is not simply regime switching

A skeptical student may ask whether the architecture is merely a complicated version of a regime-switching strategy. The question is valuable because it forces us to identify what is new.

A hard-coded regime-switching strategy typically defines a fixed set of states and a fixed mapping from each state to a trading rule. For example, if volatility exceeds a threshold, reduce risk; if trend is positive, use momentum; if trend is negative, use mean reversion. The mapping is part of the original strategy design.

Recursive Organizational Adaptation separates state diagnosis, proposal, experimental validation, and governance. The Supervisor may propose more than one specialist. The sandbox can reject the proposal. The current organization can remain unchanged even when a diagnostic condition appears unusual. A specialist can be retired based on evidence. The active organization therefore emerges from an experimental process rather than a direct state-to-rule lookup.

The difference is modest in the current notebook because the registry is small, but it becomes substantial as the candidate space grows. A future system might propose novel combinations, change routing among agents, or create temporary research teams. The organizational layer can therefore become a search problem rather than a state machine.

This is precisely where the connection to self-improving systems becomes strongest. Once the

organization must search among many possible organizations, the procedure used to generate and evaluate candidate organizations can itself become an object of improvement. At that point, Recursive Organizational Adaptation begins to approach a financial version of scaffold optimization.

24 Connecting Recursive Organizational Adaptation to STOP

The earlier STOP framework provides a useful conceptual vocabulary for extending the notebook. In STOP, an improver generates candidate solutions and uses an external utility function to determine which candidates survive. The improver itself can then become the object of optimization [7]. Our trading notebook already contains a similar division of labor at the organizational level.

The Supervisor generates candidate organizational changes. The sandbox evaluates them. The utility function scores them. The governance gate selects or rejects them. This is already an improver for trading organizations. What remains fixed is the way candidates are generated and tested.

A next-generation notebook could therefore introduce an outer loop that improves the organizational adaptation procedure itself. It might adapt the diagnostic variables, the proposal budget, the size of the candidate set, the observation horizon, or the minimum evidence threshold. That would move the system from *organization adaptation* to *adaptation of the organization-adaptation process*.

Such a step should be taken carefully. The current notebook's fixed governance is a pedagogical strength. If every part of the process becomes mutable at once, students lose the ability to identify why behavior changed. A sensible curriculum therefore introduces recursion gradually: first adaptive parameters, then adaptive capabilities, then adaptive organization, and only later adaptive adaptation procedures.

The conceptual destination is powerful. A financial AI system could eventually learn not only which specialists it needs but which method of diagnosing, testing, and governing specialists works best under different classes of market problems. That is a genuine form of institutional self-improvement.

25 The classroom journey: from quantitative model to governed AI institution

The notebook is most valuable when taught as a sequence of conceptual thresholds rather than as ten pieces of Python. Cell 1 establishes reproducibility and the boundary around the LLM. Cell 2 creates a world in which adaptation can matter. Cell 3 teaches the student to describe environmental differences before prescribing solutions. Cell 4 turns a strategy into an organization with roles and constitutional rules. Cell 5 creates the counterfactual fixed institution. Cell 6 transforms evidence into an organizational diagnostic. Cell 7 introduces

flexible AI reasoning. Cell 8 insists on experiment before deployment. Cell 9 makes the loop recursive. Cell 10 evaluates both performance and governance.

This sequence mirrors the intellectual transition the course is trying to produce. A student begins with a model-centric mindset: “which signal should I use?” The notebook then encourages a systems question: “which capability should be active?” Finally, it introduces an institutional question: “what process should decide whether a capability may enter or leave the organization?”

The progression can be summarized as:

**Model → Agent → Constellation → Feedback → Organizational Adaptation →
Governed Recursive System.**

This ladder is a more meaningful way to introduce artificial intelligence into Algorithmic Trading than simply replacing a regression with an LLM. The LLM’s comparative advantage in the notebook is not arithmetic. It is flexible reasoning about system state. The financial models retain responsibility for the quantities they calculate well. The governance architecture retains responsibility for authority.

A useful classroom discussion is to ask where students would move the boundary if real capital were introduced. Should the Supervisor be allowed to activate a volatility specialist automatically? Should a liquidity specialist require human approval? Should any AI agent be allowed to propose a change in leverage? Should a constitutional amendment be allowed if a human committee signs it? These questions transform the notebook from a coding exercise into a model of future financial governance.

26 Research extensions for a second-generation notebook

The minimalist notebook intentionally restricts the organizational search space. A second-generation version could expand in several directions without abandoning the core architecture.

The first extension is multi-asset trading. Instead of one synthetic price series, the market could contain equities, rates, foreign exchange, commodities, and credit factors with regime-dependent correlations. Specialist agents could focus on correlation breakdown, concentration, factor crowding, or hedging. The Supervisor would then reason about portfolio organization rather than a single signal pipeline.

The second extension is richer execution. Synthetic order books or market-impact functions could create environments in which execution strategy is genuinely endogenous. A liquidity specialist could choose among pacing styles, while a separate execution-risk agent monitors slippage and capacity. Governance could restrict which venues or order types are permitted.

The third extension is learning from the audit ledger. Rather than sending only current diagnostics to Claude, the Supervisor could retrieve similar historical episodes and compare the outcomes of previous organizational interventions. This would create case-based institutional memory.

The fourth extension is adversarial validation. A Challenger Agent could be tasked with finding reasons why the proposed reconfiguration might fail. The sandbox could include stress scenarios

constructed specifically to attack the proposal. Admission would require surviving both the advocate and the challenger.

The fifth extension is budget-aware organization. Every specialist could have a compute cost, latency cost, API budget, and governance cost. The utility function would then optimize institutional value net of operating expense. This would make organizational simplicity economically rather than merely conceptually valuable.

The sixth extension is human-AI governance. Adaptations could be classified by materiality. Low-materiality changes would remain automatic, medium changes would require a human click, and high-materiality changes would generate a structured memo for a risk committee. This would connect the notebook directly to real financial governance workflows.

The seventh extension is meta-improvement. Once the basic organization is understood, the system could experiment with alternative adaptation rules, observation horizons, candidate-generation strategies, or utility weights. This would bring the STOP idea fully into the organizational layer while preserving a higher constitutional boundary.

These extensions suggest that Recursive Organizational Adaptation is not one strategy. It is a research program about how AI systems can reorganize financial cognition under evidence and control.

27 Limits of the current notebook

A self-contained teaching paper should be explicit about what the notebook does not establish. The market is synthetic and deliberately structured. The regime boundaries are known to the instructor. The specialist registry is tiny. The backtest uses the same observation window both to diagnose and to compare candidate organizations. Real markets would require more rigorous out-of-sample protocols.

The LLM's action space is highly constrained. This is intentional, but it means the system is not creating fundamentally new trading capabilities. It selects among pre-coded specialists. The term "self-reconfiguring" is therefore more accurate than "self-programming."

The utility function is illustrative. Its coefficients have not been economically calibrated. Real institutions would need utility measures aligned with capital constraints, business objectives, risk appetite, and regulatory obligations.

The regime cycle is coarse. A real system would not wait for a neat 180-day block and then use exactly sixty days for observation. Change points are uncertain. Adaptation decisions would themselves need timing logic.

The Supervisor's rationale can be helpful pedagogically but should not be confused with causal proof. Language models can produce persuasive explanations. The sandbox and governance gate exist precisely because natural-language reasoning is not enough.

Finally, the notebook does not solve the problem of distribution shift. A specialist that looked useful in the synthetic family may fail in real data. The architecture demonstrates how one might govern adaptation, not that the current specialists are suitable for live trading.

These limitations do not weaken the educational objective. They define it. By holding many

dimensions fixed, the notebook isolates one concept: the organization itself can become adaptive while authority remains bounded.

28 Conclusion: from trading algorithms to adaptive financial organizations

The most important change introduced by this notebook is not a new forecasting model. It is a new unit of analysis. Traditional algorithmic trading asks whether a model produces useful predictions and whether those predictions can be converted into profitable, risk-controlled trades. Recursive Organizational Adaptation asks an additional question: is the organization that produces those predictions and controls those trades itself appropriate for the current environment?

The notebook makes this question concrete. A synthetic market creates six distinct problems. A fixed base organization confronts all six with the same structure. Diagnostic measurements reveal when volatility, liquidity, or serial dependence depart from ordinary conditions. A language-model Supervisor interprets those measurements and proposes a bounded organizational response. A sandbox tests the proposal against alternatives. A governance gate authorizes change only when measured improvement is sufficient and constitutional limits are respected. The deployed organization becomes the starting point for the next cycle, making the process recursive.

This is a deliberately conservative form of self-improvement. The system does not rewrite its own model weights. It does not execute arbitrary LLM-generated code. It does not change its leverage cap. It does not expand its own permissions. The intelligence of the architecture lies in the separation of roles: deterministic financial models calculate, the LLM reasons about organization, the sandbox evaluates, and the constitution governs.

For students, this separation offers a durable framework for thinking about the next generation of algorithmic trading. Artificial intelligence need not be treated as a mysterious predictor that replaces traditional quantitative methods. It can be used as a layer of organizational reasoning above them. The future trading system may contain many familiar models, but those models may live inside an institution that can recognize when its current capabilities are inadequate, create a temporary specialist, validate the change, monitor the result, and later return to a simpler form.

The broader implication extends beyond trading. Financial institutions are organizations of specialized cognition. Research, portfolio construction, risk, compliance, operations, and execution are different capabilities coordinated under governance. As AI begins to participate in these roles, the architecture of the institution becomes programmable. Recursive Organizational Adaptation is a way to think about that programmability without surrendering control.

The final lesson can therefore be stated simply:

Yesterday we programmed the trading strategy. Today we can build agents that operate the strategy. Tomorrow we may build governed organizations that redesign the division of intelligence itself – but only when evidence justifies the change and the constitution permits it.

That is the real transition from traditional algorithmic trading to AI-native financial systems. It is not the abandonment of models. It is the embedding of models inside organizations that can learn how to organize themselves.

A Appendix A: A compact glossary for students

Term	Meaning in this notebook
Model	A numerical procedure that transforms data into an estimate, signal, risk measure, or decision variable.
Agent	A component with a defined role, inputs, outputs, and authority. The agent may contain deterministic logic or language-model reasoning.
Base organization	The Signal, Risk, Execution, and Supervisor roles before optional specialists are activated.
Specialist	A pre-approved capability that modifies one narrow part of the organization, such as risk scaling, execution pacing, or signal composition.
Supervisor	The meta-agent that diagnoses organizational mismatch and proposes bounded additions or retirements. It does not trade directly.
Diagnostic state	A compact representation of recent market conditions, current specialists, and organizational performance.
Sandbox	The deterministic environment in which candidate organizations are compared before deployment.
Organizational utility	A score combining risk-adjusted performance, drawdown, turnover, cost, and complexity.
Governance gate	The deterministic admission rule that authorizes or rejects organizational change.
Constitution	The set of non-negotiable rules defining the specialist registry, leverage cap, complexity limit, minimum improvement threshold, and limits on LLM authority.
Recursive adaptation	The process by which the organization produced by one cycle becomes the starting organization for the next cycle.
Audit ledger	The record of what the organization was, what was proposed, what was tested, what was authorized, and what was deployed.
Self-improvement	Improvement in overall system behavior caused by changing the organization of approved capabilities, not by changing model weights or rewriting unrestricted code.

B Appendix B: Notebook-to-paper map

Notebook cell	Architectural role	Question for the student
Cell 1	Laboratory and Claude interface	Where does probabilistic AI enter, and what remains deterministic?
Cell 2	Synthetic market	What environmental changes could justify different capabilities?
Cell 3	Environmental inspection	Can we describe the problem before prescribing an organizational response?
Cell 4	Base agents, specialists, constitution, utility	What is the organization, what may change, and what may not change?
Cell 5	Fixed benchmark	Where does the unchanged organization fail or incur unnecessary cost?
Cell 6	Diagnostic state and fallback supervisor	How do observations become an organizational hypothesis?
Cell 7	Claude bounded supervisor	How can flexible AI reasoning contribute without acquiring execution authority?
Cell 8	Sandbox and governance gate	What evidence is required before an organizational proposal becomes operational?
Cell 9	Recursive loop	How does one adaptation become the starting state of the next?
Cell 10	Final comparison and controls	Did adaptation improve the institution without violating its constitution?

SUGGESTED CLASSROOM EXERCISE

Ask students to predict, before running Cells 6 through 9, which specialist they expect to be useful in each regime and which specialist they expect to be retired during recovery. After the notebook runs, compare the students' organizational hypotheses with the Supervisor's proposal, the sandbox winner, and the final governance decision. The objective is not to guess perfectly; it is to distinguish diagnosis, proposal, evidence, and authority.

C Appendix C: Figure file map for Prism/OpenAI LaTeX editor

Place the nine infographics in a folder named **figures** beside this **.tex** file. Use the exact filenames below. The document already contains calls to each image and will replace placeholders automatically when the files are uploaded.

Filename	Content
figures/cell_2.png	Synthetic market with six structural regimes.
figures/cell_3.png	Inspection of regime statistics and segmented price path.
figures/cell_4.png	Base organization, specialists, constitution, backtester, and utility.
figures/cell_5.png	Fixed-architecture benchmark and regime-by-regime performance.
figures/cell_6.png	Diagnostic state and transparent fallback supervisor.
figures/cell_7.png	Claude Sonnet 5 as bounded organizational supervisor.
figures/cell_8.png	Sandbox validation and governance gate.
figures/cell_9.png	Full recursive organizational adaptation loop.
figures/cell_10.png	Final comparison, audit summary, and control checks.

References

References

- [1] A. Air, Reworr, N. Kotov, D. Volkov, J. Steidley, and J. Ladish. *Language Models Can Autonomously Hack and Self-Replicate*. Palisade Research, 2026.
- [2] R. Almgren and N. Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39, 2001.
- [3] J. D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.
- [4] A. W. Lo. The Adaptive Markets Hypothesis: market efficiency from an evolutionary perspective. *Journal of Portfolio Management*, 30(5):15–29, 2004.
- [5] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*, 2015.
- [6] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018.
- [7] E. Zelikman, L. Lorch, L. Mackey, and A. T. Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. In *Conference on Language Modeling (COLM)*, 2024.
- [8] Anthropic. Claude platform documentation: Messages API and model usage guidance. Documentation referenced by the companion Colab notebook, 2026.